# Hydrofrac: what ships, what we learned, which doors are closed

*An inventory and retrospective of lineage-driven fault injection for Hydro, written against commit `2f156b6335` (bookmark `sandbox-6479bd3c-b09a-4092-b96d-1375d67833f6`, 2026-10-20).*

Hydro's type system certifies consistent computation: a collection labelled `EventualConsistency` is one whose contents at every cluster member are a deterministic function of the set of inputs that member received, insensitive to order and retries. The label is silent about whether the input sets are the same. A broadcaster that dies between the second and third send of one fan-out leaves two members with different inputs, and each member's contents are then perfectly consistent with its own inputs and different from its neighbour's. Hydrofrac discharges that missing premise. It runs the program in the Hydro simulator, injects the crashes and omissions the program's own transport declarations admit, and searches, lineage-first in the manner of Molly (Alvaro, Rosen, Hellerstein, SIGMOD 2015), for an execution in which a fact held by one correct member at the end of the run is absent at another. What converged, after six rounds, is a two-level tool (`Level::Execution` and `Level::Precise`), Molly's fault model (every failure at a time at or before the end of finite failures, enforced by the simulator), fault points at the sender on every network send, and a fault universe that is grown lazily from the derivations the search has seen. The last round's experiments — a four-level lineage ablation, disjunctive provenance, aggregate-local witnesses — mostly closed doors rather than opening them, and that is why I believe the design has converged rather than merely stopped.

---

## Part I — The artifact as it ships (`2f156b6335`)

### 1. What it checks and how to run it

The automatic path is `Spec::all_labels` plus `Ldfi::search_all_labels`. The test supplies a flow, cluster sizes, the crashable nodes, a crash budget and a body that drives inputs and timers; hydrofrac finds every IR node where the `EventualConsistency` label is minted or exits the labelled region, taps each, and runs one search per collection. Nothing is named and no property is written (hydrofrac.md §"Using hydrofrac"):

```
let mut flow = FlowBuilder::new();
let source = flow.process::<Source>();
let members = flow.cluster::<Member>();
let (send, items) = source.sim_input::<u32, TotalOrder, ExactlyOnce>();

let delivered = reliable_broadcast_closed(items, &members);   // labelled EventualConsistency
delivered.weaken_consistency().sim_cluster_output();

let spec = Spec::all_labels(&mut flow);
let compiled = flow.sim()
    .skip_consistency_assertions()
    .with_cluster_size(&members, 3)
    .with_provenance()
    .compiled();

let results = Ldfi::new()
```

```
    .level(Level::Precise)
    .crashable("source", &source)
    .crashable_cluster("member", &members, 0..3)
    .crash_budget(1)
    .spec(&spec)
    .search_all_labels(&compiled, async |_run| {
        send.send(1);
        send.send(2);
        hydro_lang::sim::quiesce().await;
    });
println!("{results}");
```

It prints one line per labelled collection: the source's single-hop broadcast violated by `crash(source after … [→1]:2)` in 5 hypotheses, the members' echo broadcast violated by an echoer's crash in 3, and the delivered set certified in 10 hypotheses under exhaustive schedules (hydrofrac.md §"Using hydrofrac"). `results.verdicts` holds one `LabelVerdict` per collection, `Searched(outcome)` or `NotChecked(reason)`; tests assert with `results.at("file.rs:149")`, `results.all_certified_or_not_checked()` or `results.counterexamples()`.

The generated specification, `Spec::eventual_consistency`, is a universal over a fixed fact domain: a failure-free probe collects every fact seen at any member into `FACTS`, and under every later hypothesis each fact of `FACTS` present at some correct member must be present at every correct member. A fact absent everywhere creates no obligation; a probe that yields no facts is `Vacuous` (hydrofrac.md §"What hydrofrac checks").

A property the types do not express is written by hand: `Spec::new(&mut flow)` gives an observer process; `spec.observe(..)` / `observe_cluster(..)` / `observe_sequenced(..)` ship an output stream to it; `spec.correct_members(&cluster, 0..n)` is the complement of the hypothesis's crash set; `spec.pre(view)` and `spec.post(view)` are bounded `Optional`s in the observer's tick, and a run with `PRE` non-empty and `POST` empty is a counterexample. `Ldfi::search` runs one such specification. The older goal API (`goal`, `goal_all_members`, `goal_witnessed`) is still compiled (ldfi.rs module docs §"Goals").

`Ldfi::new()` defaults to `Level::Precise`, `Exploration::Exhaustive`, `Strategy::Lineage`, a crash budget of 0, a hypothesis budget of 256 and a witness cap of 8 (ldfi.rs `Ldfi::new`). The knobs and their environment overrides:

| Knob | Default | Env | Effect |
|---|---|---|---|
| `.level(Level::Execution \| Precise)` | `Precise` | `LDFI_LEVEL=exec\|precise` (`record` is an alias) | which lineage oracle derives a record (commit fafcd70fe3) |
| `.disjunctive_provenance(bool)` | off | `LDFI_DISJUNCTIVE=1` via `.disjunctive_from_env()` | follow every witness of a deduplicated record (commit 73a5db9c05) |
| `.aggregate_witnesses(bool)` | off | `LDFI_AGGREGATE=1` via `.aggregate_from_env()` | minimal passing input subsets at threshold folds (commit 2f156b6335) |
| `.max_witnesses(k)` | 8 | — | cap on witnesses per record; overflow falls back to the union witness, soundly |

| Knob | Default | Env | Effect |
|---|---|---|---|
| `.with_traces()`, `.with_provenance_graphs()` | off | — | retain every schedule's `Trace`; record a `ProvGraph` beside each derivation (viz) |
| `.exploration(Exploration::Fuzz { instances })` | `Exhaustive` | — | sampled schedules; a certificate is then not a proof |
| `.strategy(Strategy::Blind)` | `Lineage` | — | the ablation: same universe, no clauses |
| `.max_hypotheses(n)` | 256 | — | `Outcome::Budget` when exceeded |
| gallery tests | no-op | `MOLLY_VIZ_OUT=<dir>` | `gallery_*` tests write `.dot` and `report.md` (gallery README) |
| ablation harnesses | no-op | `LDFI_ABLATION_OUT=<path>` | `level_ablation_*` tests write their Markdown grid (ablation header) |

`LDFI_PROV_DEBUG`, `LDFI_SOLVER_DEBUG`, `LDFI_LINEAGE_DEBUG` and `LDFI_REPLAY_DEBUG` print, respectively, records with more than one witness, every clause and hypothesis, the lineage log and the replays (ldfi.rs). Building with the `ldfi` feature needs cmake and libclang for the HiGHS bindings (commit d34bd78add).

## 2. The fault model as shipped

Every failure happens at a time `≤ EFF`. The body declares the end of finite failures with `run.end_of_finite_failures()`; from then on omissions stop applying and no crash point fires. This is Molly's model and, since commit `4ddea33f19`, the simulator's filter (`FaultState::filter` in `sim/runtime/lineage.rs`) enforces it for both kinds of fault, and the search's kill relation (`Dependency::killed_by`) takes the run's pre-`EFF` send counts so that what the solver believes a crash kills is exactly what the simulator does (hydrofrac.md §"Transports supply the fault model").

The fault point is a filter the simulator inserts before every `Network` send; every channel is a fault point by construction and nothing is registered. The transport type is the fault model: a `lossy_*` channel admits **omission**, identified by `(channel, sender, destination, wire bytes)` — every byte-identical copy on that channel is dropped while finite failures last, so a retransmission with the same payload is the same fault; a `fail_stop` channel admits only its **sender's crash** (ldfi.rs module docs §"Fault vocabulary").

A crash is `crash(node after m)` or `crash(node before any send)`: the node performs every send up to and including `m`, every later send on every channel is dropped, and its external outputs are suppressed. Naming the point by a message rather than a send count keeps the hypothesis meaningful across runs whose send sequences differ, and lets one crash split the fan-out of a single broadcast. Only nodes registered with `.crashable` / `.crashable_cluster` may crash, within the budget. A crash point is named by a pre-`EFF` send of the run that minted it; in a run where that message is sent only after `EFF`, or never, the node survives (hydrofrac.md, same section).

So `EFF` placement decides what is testable. The ISR harness declares `EFF` after the first round of replica timers so that the leader's acknowledgement and heartbeats lie inside the window; with `EFF` right after the client's write, the leader's post-acknowledgement crash is inadmissible and the unclean-election bug is outside the bounds (commit 4ddea33f19). The window after `EFF` is the recovery window, and a body that ends faults without firing the protocol's

retransmission, gossip or election timers afterwards has not asked a convergence question. Where `EFF` falls also decides which messages are omission candidates: Paxos's P1b-counting bug became visible only when the body moved `EFF` past the first ballot (history §9.2).

## 3. Verdicts

| Verdict | What it is worth | What it does not imply |
|---|---|---|
| **Certified** (`Outcome::Certificate`, `Exploration::Exhaustive`) | No admissible fault set, under any schedule, breaks the generated check within this test's topology, workload, horizon, crash budget and `EFF`. | Nothing about other inputs, cluster sizes or longer horizons; nothing about convergence beyond the horizon. |
| **Certified, RELATIVE** (`Grade::HandWritten`) | No hitting set remained; the recorded derivations persisted under every excluded fault set. | Soundness of the inference for the view: a hand-written `POST` falsified by an *extra* fact (agreement, uniqueness, a final value) can be missed (ldfi.rs `Grade`). |
| **Violated** (`Outcome::Counterexample`) | A concrete fault set and a concrete schedule under which `PRE` held and `POST` did not. One failing run suffices; exhaustive exploration is not required. | That the protocol is wrong rather than the horizon too short; the trace must be read. |
| **No violation under Fuzz** | `instances` sampled schedules per hypothesis passed and no hitting set remained. Evidence. | A certificate; a violation in an unsampled schedule is missed. |
| **Budget** (`Outcome::Budget`) | The hypothesis cap was reached. | Anything about the protocol. |
| **Vacuous** (`Outcome::Vacuous`) | The failure-free run never satisfied `PRE`: the harness protects nothing. | Anything about the protocol; a statement about the body. |
| **Not checked** (`LabelVerdict::NotChecked(reason)`) | The collection is outside the generated specification's class: local inputs only, not a stream, inside a tick, excluded by the test. | Silence: the reason is printed, and nothing fails for a label the test does not assert on. |

(hydrofrac.md §"Judgements"; ldfi.rs `Outcome`, `Grade`.) `Strategy::Blind` ends in `Outcome::Exhausted` rather than a certificate.

## 4. What it has found

**Real defects in production code**, all from the hand-written harnesses (history §9.2; hydrofrac.md §"Hand-written specifications"):

- A message loop in Paxos's leader change: a new leader re-sent its recommits on every later tick and each round of P2bs triggered the next, so a partially replicated slot made proposer and acceptors exchange the same messages forever. Surfaced as the first faulty hypothesis never quiescing.
- Compartmentalized Paxos's read quorum was a row, not a column: any `rows` P1bs of the grid elected a leader, which does not intersect the other row's write quorum. A violation of agreement on the *correct* variant in 8 hypotheses, with no fault, only a P1b ordering.
- Paxos's P1b quorum counted replies, not distinct acceptors: a re-sent P1a drew a second P1b from the same acceptor, and two replies from one acceptor made a quorum. Violation on correct Paxos in 48 hypotheses and on correct compartmentalized Paxos in 18, once `EFF` was moved past the first ballot.

- `collect_quorum_with_response` was not a function of its input despite its `nondet!` justification; an idempotence-justified `fold` lacked a simulator implementation of its retry-only cast. Both found by the simulator on the way.
- Hydro's Maelstrom backend shared one untagged message type across every cluster-to-cluster channel, so two such channels deserialized each other's bytes. Found by the first Maelstrom run of the Raft comparison (history §9.1).

**Injected bugs.** Raft's election restriction with index-only log comparison passes all thirteen of `raft.rs`'s behavioural tests and is found in 90 hypotheses with four omissions and no crash, a scenario I had not written down: member 0 wins term 1 and appends `r0`, lost on both links; member 1 wins term 2, appends `r1` at the same index and commits it, its replication to 0 lost twice; in the heal phase 0 campaigns with `(index 1, term 1)`, wins under the index-only check and overwrites the committed `r1` (history §9). Correct Raft certifies RELATIVE in 235 hypotheses under the same body. ISR's unclean election is `{omit(repl[0→1]), crash(replica@0 after client-ack)}` in 9 (closure) or 12 (dataflow) hypotheses at `Precise`, 14 at `Execution` (ablation §"ISR replication"). Plain 2PC breaks under one coordinator crash between two `commit` sends in 2 hypotheses at every size; 2PC with cooperative termination certifies under one crash in 9 / 13 / 18 / 23 hypotheses at `n=2/2/3/4` (loads 1/ ≥ 4/ ≥ 4/ ≥ 4) at `Precise`, and breaks under two (ablation §"2PC"; hooks §"Prototype status"). Core Paxos's commit quorum one short is found with two P2a omissions, and its correct variant certifies at `Precise` in 230 hypotheses within a cap of 256 where `Execution` runs out of budget (ablation §"Final: two levels", Raft/Paxos table).
`hydro_std::quorum::collect_quorum` certifies for a deduplicating caller in 28 hypotheses and breaks for the naive one in 5 (history §9).

**The `EventualConsistency` corpus**, one crash allowed (hydrofrac.md §"Results on existing Hydro code"):

| Program | Result |
|---|---|
| `broadcast_closed`, process → cluster, `fail_stop` | Violated, 5 hypotheses: `crash(source after bcast[→1]:Item(2))` |
| `broadcast_closed`, process → cluster, `lossy_delayed_forever` | Violated, 2: `omit(bcast[→0]:Item(2))` |
| `broadcast_closed`, cluster → cluster, `fail_stop` | Violated, 5 |
| `broadcast_live_from_process` | Violated, 5 |
| `leader_merge_broadcast` | Violated, 2: a leader crash mid-fan-out |
| Echo reliable broadcast; `reliable_broadcast_closed`, `_live` | Certified, 10 hypotheses each, exhaustive |
| `uniform_reliable_broadcast_closed`, thresholds 1 and 2 | No violation, 28 hypotheses, 8 sampled schedules |
| `g_set_gossip`, `_live` | No violation, 13, 8 sampled schedules |
| `multi_paxos` learners' `learned`, 1 proposer, 3 acceptors, 3 learners | No violation, 24–26, 8 sampled schedules |

Under `Spec::all_labels` multi-Paxos reports ten collections: every minted single-hop `broadcast_closed` from the proposer (`:281`, `:549`, `:582`) is violated by the proposer's crash mid-fan-out in 5–9 hypotheses, the learners' echo keyed by echoer (`:591`) by a learner's crash in 10, the delivered `learned` set (`:588`) and its `SpliceFact` map (`:608`) pass in 25–26, and the `fold` into a `Singleton` at `epoch_splice.rs:190` is not checked (hydrofrac.md §"Every label of multi-Paxos"). The minted transport grant is violated and the delivered set after the echo passes; only the search separates the two.

**An honesty note.** An earlier version of the Raft result reported the election-restriction bug found in 17 hypotheses. That number came from a body that staged the bug's scenario by hand — elected member 1, raced member 0's timer

against a request, fired member 1's timer twice after `EFF` — so the search only had to find two omissions inside a scenario its author had arranged, and the Maelstrom comparison credited lineage for conditions the body had set up. I wrote that body and reported the number without saying so. It is withdrawn, with the 25-vs-160-vs-random comparison it fed; the 90 / 235 figures are from a generic body shaped only by what *correct* Raft needs to converge (history §9).

## 5. Visualizations

A violation you cannot read is not useful, so `hydro_lang::sim::viz` (feature `ldfi`) draws Molly's two zoom levels and one more (molly-viz.md).

*Zoom 0 and 1: Lamport diagrams.* A `Trace` is the serializable record of one run: nodes with their crash point, events in global scheduling order, messages in send order with fate (`Delivered | Omitted | Crashed`), and the `EFF` cut. It keeps no payload bytes; a message matches a `Dependency` by `(channel_id, from, to, FNV-1a hash of the bytes)`. A failing schedule always retains its trace; `Ldfi::with_traces()` retains every schedule's, which the witness picture needs. `lamport::mermaid` emits a `sequenceDiagram` (`->>` delivered, `-x` omitted, `--x` lost to the sender's crash, 🔢 and `EFF` notes, ★ on highlighted messages); `lamport::dot` emits a pinned-position `neato` spacetime diagram (a lane per node, one rank per event on an invisible time spine, red dashed `✗` for omissions, gray dashed for crashed sends, ☠ `crashed` ending a lane, a dashed `EFF` rule; commit c3164dd837 added self-placed labels that overlap nothing across the fifteen gallery files). `Outcome::viz()` returns a `Counterexample` with `lamport_*()` for the failing run, `witness()` for the last goal-held schedule with a trace, `support_lamport_*()` with `Trace::support(derivation)` highlighted, and `markdown()` for both as a report (commits 18fdd58cec, c3164dd837).

*Zoom 2: the why-provenance graph.* `Ldfi::with_provenance_graphs()` records a `ProvGraph` beside every derivation: `Goal`, `Output`, `Exec`, `Message`, `Alive`, `Base` and `Opaque` nodes; `Checked`, `Produced`, `Consumed`, `Carried`, `Derived`, `Counterfactual`, `Fallback` and `Alive` edges, each tagged `exec`, `record` or `replay`, so a `Precise` graph that hit an opaque record shows the `Opaque` node and the `Replay` edges beneath it. Recording never changes a derivation; `ProvGraph::dependencies()` equals the flat `Derivation`, which `ldfi_viz::provenance_graphs_agree_with_derivations_at_every_level` checks at both levels. `collapsed()` elides executions for the compact goal → messages picture (commit 567798e8ea; molly-viz.md §"Zoom level 2+").

*The gallery* (`docs/design/molly-viz-samples/`) has five directories — `two_pc`, `isr_unclean_election`, `paxos_commit_quorum`, `reliable_delivery_gives_up`, `plain_broadcast` — each with `failing`, `witness_{execution,record}` and `provenance_{execution,record}` as `.dot` and `.png`, plus `report.md`. Regenerate with `MOLLY_VIZ_OUT=/tmp/gallery cargo test -p hydro_test gallery_ -- --test-threads=1`, then `neato -n2 -Tpng` for the Lamport files and `dot -Tpng` for the provenance graphs (gallery README). The ISR report shows the omitted `repl[0→1]`, the shrink, the acknowledgement, the 🔢, the `EFF` rule, and ZK's promotion of replica 1 beneath it.

*Known limits.* External outputs render as a hex prefix; nodes the fault model does not name appear as `Process(locNvM)`; operators a record traversed are not tracked, so the graph shows which messages a fact rests on, not the `join` that combined them; alternative derivations of a deduplicated record are not drawn (molly-viz.md §"Limits").

### 6. Explicit non-coverage and cost

Hydrofrac runs only in the simulator; the filter is a simulator artefact, not an IR node, so nothing exists under `deploy` (history §10, §11.3). The fault vocabulary is omission and crash: no delay, no reordering (the schedule explorer's dimension), no partition primitive (omission sets express one), no Byzantine behaviour, no duplication (hydrofrac.md §"Limitations"). Embedded channels are not fault points (history §5.6). Scalar labelled collections (`Singleton`, `Optional`, `KeyedSingleton`), collections minted inside a tick, and collections with local inputs only are reported and not checked; an element type that is not `Clone + Serialize` fails the generated crate's compilation and has to be excluded with `Spec::all_labels_except` (hydrofrac.md §"Which collections are checked"). Membership is static. A certificate is for one test, one topology, one workload, one horizon.

Cost, on an 8-core debug build: ISR unit searches take seconds to half a minute (ablation §"ISR replication"); the 2PC level grid of 144 searches takes 162 s (ablation §"2PC"); echo broadcast at `n=3`, two crashes, exhaustive, 52–76 s, while `n=4` exhaustive is 92,160 schedules per hypothesis (~5 min each) and runs under `fuzz 8` (ablation §"Reliable broadcast", §"Exhaustive feasibility probes"); the Paxos `Precise` certificate at 230 hypotheses took 116 s with one sampled schedule per hypothesis and 1143 s at 393 with four (ablation §"Final", §"Aggregate-local counterfactual witnesses"); Raft at `Precise` runs 4–5 s per hypothesis and the full durability searches take about thirty minutes for the bug and an hour for the certificate (ablation §"Raft"; history §9); the three corrected Paxos certificates (372, 301, 568 hypotheses) take about forty-five minutes together (hydrofrac.md §"Hand-written specifications"). Regression suites: `cargo test -p hydro_test ldfi -- --test-threads=1` (56 passed, 12 long certifications ignored, ~12 min) and `cargo test -p hydro_lang --features ldfi --lib sim` (233 passed) (commit 2f156b6335; ablation §"Regression suites").

---

## Part II — What we learned

### 1. Provenance granularity: which levels earned their keep

The question was whether finer lineage buys fewer hypotheses, and how much finer it needs to be. At the start of the last round the code had four levels: `Execution` (L0: every output of an execution depends on every input of it and of every earlier execution at its node), `Slice` (L1: L0 narrowed by a static backward slice from the producing sink, then the default), `Record` (L2: record provenance from the `with_provenance()` IR rewrite, each element carrying the delivery ids it was computed from, replaying the node at a `Prov::OPAQUE` record), and `Counterfactual` (L3: node-local replay with deliveries removed one at a time, latest first, keeping those without which the record disappears) (ablation header). The experiment ran the *same* search at all four levels on the same provenance-compiled program, over 2PC (`n ∈ {2,3,4}` × load `{1,4,8}` × crash `{0,1}` × CTP), ISR (two encodings × bug/correct × three explorations), reliable broadcast and delivery, Raft and core Paxos (commits c11ac0147a, f956b8c5b4, 6b2acd41d8, 31d3eec1e8).

*L0 equals L1 in every cell of every protocol.* On 2PC the columns agree across all 36 configurations; on ISR in every cell of both encodings; on broadcast and delivery in every row; on Raft to the digit (ablation §"Observations (2PC)", §"ISR", §"Reliable broadcast", §"Raft"). The slice never separated a record from the inputs of its execution. On a closure-encoded protocol it cannot — `raft_step` is one IR sink — and on the dataflow encodings both inputs of a member's tick feed the same outputs.

*L2 pays exactly where there is background load or redundancy.* On 2PC+CTP, L2's derivation stays at `4·n` dependencies and its certificate count is load-invariant at 13 / 18 / 23 hypotheses for `n = 2 / 3 / 4`, while L0's grows about nine hypotheses per background transaction, to 51 / 76 / 101 at load 8: 101 → 23 at the largest cell (ablation §"2PC with cooperative termination"). The earlier "11 vs 73" reproduces as 18 vs 76 — 4.2× rather than 6.6× — deterministic over five repeats; the shift is uniform across levels and loads and is probably the harness, but this was not verified (ablation §"Observations (2PC)"). On ISR, clean, one crash: 31 → 12 (closure) and 31 → 18 (dataflow); on the bug 14 → 9 / 12. On echo broadcast: 15 → 10 at one crash and 73 → 32 at two, with 20–30% less wall time despite the bookkeeping, because execution lineage charges a member's delivery to *every* message in its tick batch (ablation §"Reliable broadcast"). On Paxos, L2 certified the correct variant in 195 and 230 hypotheses in two passes where L0 ran out at 256 (ablation §"Paxos", §"Final"). Reliable delivery is level-insensitive: one-leaf derivations.

*L3 reproduced L2 wherever L2 applied, at two to ten times the cost.* On 2PC, L3's count equals L2's on every certificate at 3–10× the wall time (11.2 s against 1.3 s at `n=4 load=8`); on broadcast, identical hypotheses, schedules and derivations at 2.7–3.5× (65,388 replays against 17,280); on Paxos, same-sized derivations at 1.6–2.3× (ablation §"Observations (2PC)", §"Reliable broadcast", §"Paxos"). Its one real win is the dataflow ISR bug in 3 hypotheses instead of 12, because its failure-free derivation is `{hb×3, observed}` and the leader's crash is hypothesis #2 (ablation §"ISR"). That does not pay for Raft, where replay is ~5 s a hypothesis (ablation §"Recommendation record").

*The Raft exception.* Raft is one `raft_step` closure per tick, so under the rewrite its outputs are opaque and L2 replays through the step — L2 *is* counterfactual lineage there, and L3 adds nothing. Derivations shrink 4.4× (46.4 → 10.5 leaves), but hypotheses go up: L0 certifies both variants in 59 hypotheses in 17–19 s; L2 runs out of a 64-hypothesis budget after 243–306 s and, given 160, certifies in 107 after 403 s. The over-approximate clause "everything delivered to the member" is hit by almost any fault set, so the universe saturates fast; the precise clauses each name a different path and must be defeated separately (ablation §"Raft"). A program that is closure-encoded end to end should pick `Execution` explicitly, and that is why `Execution` survives as a level (ablation §"Recommendation record").

*The collapse.* Commit `4ddea33f19` removed `Slice` and `Counterfactual`, with the re-measured two-level tables agreeing with the L0 and L2 columns in every cell. Commit `fafcd70fe3` then fixed the fallback the collapse had got wrong: a program compiled without `with_provenance()` had been silently downgraded to `Execution` (`Ldfi::effective_level`), when the right reading of a program with no tags is *every record opaque*, derived by replay. `Record` became `Precise` — record provenance where the program carries it, counterfactual replay where it does not, execution closure only where replay is inconclusive — and `effective_level` was deleted. On ISR compiled without provenance: the bug in 9 hypotheses instead of `Execution`'s 14, the certificate in 12 instead of 31, zero fallbacks; on 2PC+CTP `n=3 load=8`, 18 instead of 76, with 27 fallbacks because the coordinator's transactions arrive through an un-hooked `sim_input` and a solo replay never sends a `prepare` — the fallback doing its job; on `leader_merge_broadcast`, which the rewrite rejects, the same 2-hypothesis counterexample with 14 replays, 0 fallbacks and no pinned level (ablation §"Precise without provenance"). Replay is an oracle inside `Precise`, not a level: it runs at opaque records, at every record of programs without provenance, and at the observer for every hand-written view, which is why the `Execution` cells of the ISR tables show replays too.

## 2. The soundness bug the ablation caught

The first ISR grid showed L2 finding the unclean-election bug while L0, L1 and L3 **certified** it, on both encodings, under `fuzz 1` and `fuzz 8`; the repo's own `ldfi_finds_the_unclean_election_bug{,_dataflow}` failed

under `LDFI_NO_PROVENANCE=1` (certificate in 14) and `LDFI_LEVEL=replay` (certificate in 6) (ablation §"Soundness bug", pre-fix table; commit f956b8c5b4). The diagnosis written down at the time — vacuous hypotheses adding no clause and being read as coverage — did not survive the solver's own trace. The counterexample `X = {omit(repl[0→1]), crash(replica@0 after client-ack)}` was a superset of none of the vacuous sets the search saw. `X` was never proposed because **`X` was not a hitting set of the clauses the search had**: some clause claimed `POST` rested only on deliveries `X` does not kill, while the simulator shows `X` kills `POST`. Two unrelated gaps produced such clauses (commit 4c43956b4b).

*Root cause 1 (L0, L1): dependency ordinals clamped at EFF.* `Lineage::crash_ordinal` clamped a dependency's send ordinal to the sender's pre-`EFF` send count, assuming crash points lie before `EFF`; the simulator fired `crash(n after m)` wherever `m` was sent. In the omission world replica 0's acknowledgement was post-`EFF` (send #4 of `#0 repl[0→1] #1 repl[0→2] | EFF | #2 shrink #3 hb #4 client-ack #5 hb …`), every replica-0 dependency carried the clamped ordinal #2, and `killed_by` computed `position(client-ack) = 4 < 2`: the crash kills nothing of the clause, while in fact it kills the heartbeats #5–#7 that keep 0 the leader. L2 found the bug only by hypothesis order: its smaller failure-free clause made the crash hypothesis #2, before any omission-world clause existed. The fix made the ordinal exact and moved the pre-`EFF` clamp to where crash points are *proposed* (ablation §"Root cause 1").

*Root cause 2 (L3): sequenced observations never traced.* `Spec::observe_sequenced` enumerates before the observer's hooked batch, so the release payload is `bincode((usize, View))` while the wire bytes are `bincode(View)`; `payload_matches` never matched a view to ZK's send — the "32 untraced deliveries" on every ISR log line. Replay takes only matched releases as removable candidates, so at L3 the views were fixed facts with no lineage: derivation `{observed[0→]#2}`, two crash points, a certificate in 6. Accepting a serialized `usize` prefix made the derivation `{hb×3, observed[0→]#2}` and the bug appears at hypothesis #3 (ablation §"Root cause 2").

The alternative fix for root cause 1 — confining crash points to the window in the simulator — was tried as a one-line experiment and made the ISR bug unreachable at every level under that harness, because the acknowledgement was post-`EFF` in the omission world. The search was first made to agree with the simulator; then `4ddea33f19` adopted the simulator-side reading as Molly's model and moved the ISR harness's `EFF` past the first round so the acknowledgement lies inside the window in every world (ablation §"Root cause 1", §"Final"). Where history §10 says `EFF` "should not shape the design further", the current design disagrees: it is the fault model.

After the fix, verdicts agree at every level in every cell: ISR bug at L0 (14), L1 (14), L2 (12), L3 (3), pinned by `isr_replication::tests::ldfi_finds_the_unclean_election_bug_at_every_level`; every 2PC verdict and certificate count unchanged; every broadcast and delivery cell identical (ablation §"After the fix"). "The verdict is independent of the lineage level" is now a test, and it is the strongest argument for having run the ablation: a level grid is a soundness oracle for free, since the levels are supposed to differ only in cost. Vacuous-superset pruning was left as it was; nothing showed it changing a verdict (ablation §"Vacuous hypotheses").

## 3. Disjunction: why Molly's disjunctive provenance bought nothing here

Molly's lineage from one good run is a formula: a conjunction within a derivation, a disjunction across the alternative derivations of one fact, and the solver prunes every fault set that leaves some alternative intact without running it. Hydrofrac kept one witness per record — at a dedup, the first copy's — and rediscovered alternatives by simulation. Commit `73a5db9c05` recovered the disjunction behind `Ldfi::disjunctive_provenance(bool)`: `Prov` became a shared, lazily resolved handle over `Leaf(delivery) | All(children) | Alts(children)`; pass-through operators share the handle, combinations build `All`, and a `map(rebox)` before every in-tick `unique` gives each

arrival a fresh `Alts` cell that `Tagged::eq` fills with the duplicate's alternatives (lazy and shared because DFIR's `unique` emits the first copy before its duplicate arrives). The formula resolves to a minimal DNF under `max_witnesses` (default 8; an overflowing conjunction falls back to the union witness, an overflowing disjunction keeps its first `cap` alternatives, both sound). With a cap of 1 the resolution is exactly the old single derivation, and `two_pc_ldfi::tests::single_derivation_search_is_unchanged_by_the_provenance_formula` pins the thirteen hypotheses of the exhaustive 2PC+CTP `n=2 load=4` search, in order, to the pre-change sequence (ablation §"What was built").

I expected this to reduce hypothesis counts. It did not, on any protocol, in any cell, in three measurement passes. On 2PC and 2PC+CTP every verdict and certificate count is identical, because no run exhibits two derivations of one fact: a participant queries its peers only after a timeout while uncertain, and the peer path fires only in worlds where the coordinator crashed, where the participant hears *only* the peer. Alternatives exist across worlds, not within a run, and Molly's single-run lineage would see what hydrofrac's does. On ISR, in both encodings, clauses per run are identical to two decimals across five repeats, which is only possible if no record resolved to more than one witness. On Paxos the P2b quorum is a `fold`, a conjunction (ablation §"Where the alternatives are", §"Complete grid: ISR, broadcast, delivery", §"Core Paxos").

Echo broadcast is the one protocol with in-run alternatives, and there the hypothesis count went **up**: 10 → 11 at one crash and 32 → 39 at two under exhaustive exploration at `n=3`, reproduced exactly in two passes; 15 → 21 (+40%) at `n=4` under `fuzz 8`, identical over five repeats, with the witness cap of 8 overflowing for the first time in the corpus (156–176 times per search; four members give up to four witnesses per delivery and $4^4 > 8$). The diagnostic, reconstructed from the searches' own records with every hypothesis re-checked as a minimal hitting set, explains the arithmetic. The disjunctive failure-free run yields 16 derivations instead of one, and its clauses prune the two `crash(broadcaster after initial[→k])` hypotheses the single search ran — Molly's pruning, and it is real. But every alternative's deliveries become dependencies, and their senders' early sends enter the universe: `crash(member@1 before any send)`, `crash(member@2 after echo[2→0])`, `crash(member@2 before any send)`, minted by echoes from a higher-numbered member to a lower-numbered one, which never appear in any single-witness derivation because the lower-numbered member's first copy is always the broadcaster's `initial`. Saved 2, added 3; at two crashes saved 12, added 19. A blind search over all twelve fireable crash points confirmed the goal holds under every one (ablation §"Diagnostic"; commits aa10020989, 0bbf22d080).

The deeper reason is structural. Hydrofrac's fault universe is grown lazily from the derivations it has seen; a crash point is proposed only once some derivation depends on the send that names it, so a fault that kills no observed dependency is never tried. That is itself most of the pruning Molly gets from the formula. Molly's universe is every message of the run, fixed up front, so the disjunction is what keeps *its* search from running every one of them; hydrofrac gets that from the universe instead. The DNF certificate is relative to a larger universe, so it is a stronger certificate at more runs, not a cheaper one (ablation §"Observations" under Disjunctive provenance). The flag stays off. The follow-up worth one experiment is DNF cover constraints over the single-derivation universe: keep every alternative as a clause but propose crash points only from the first-witness derivations, dropping (soundly) any clause the universe cannot hit, which would keep the two pruned broadcaster hypotheses without admitting the three new member ones (ablation §"Status").

## 4. Aggregate-local counterfactual witnesses

The disjunction only ever fired at a dedup, and the corpus's redundancy is quorum-shaped: a `fold` (a count) over `n` inputs followed by a threshold predicate, whose output is tagged with the conjunction of all `n`. "Any `k` of `n`" is lost at the aggregate, in hydrofrac as in Molly. Commit `2f156b6335` recovers it behind

`Ldfi::aggregate_witnesses(bool)` (`LDFI_AGGREGATE=1`, default off). Under the mode a `fold` / `fold_keyed` accumulator's cell is a `Node::Agg`: the conjunction of every input (what it resolves to, so every unaware reader sees the old formula) plus each input's value and provenance, up to `AGGREGATE_MAX_INPUTS = 10`, and a type-erased re-fold built from fresh instances of the fold's own `init` and `acc`. A downstream `filter` or `filter_map` re-runs the fold on all `2^n` subsets in both input orders and applies its own predicate; the output's provenance becomes the disjunction of the **minimal passing subsets** if and only if, on this run's inputs, the predicate is upward-closed, order-independent and passes the full set — checked empirically, since the IR carries no commutativity annotation. Otherwise the conjunction stays and the skip is counted by reason. The empty subset is never a witness; keyed↔flat casts re-key and project the re-folded value; every other value-changing operator strips the cell to its conjunction so no predicate re-evaluates an aggregate through a value it did not see (ablation §"What was built" under Aggregate-local witnesses). [NEEDS AUTHOR REASONING: why 10 for `AGGREGATE_MAX_INPUTS` and 8 for `max_witnesses` — measured, or round numbers?]

Where it fires: Paxos's P2b commit quorum (`collect_quorum(p2b, 2f+1)`: `fold_keyed → entries → filter_map(success >= min)`), with zero skips — but a second witness exists only in schedules that batch all three acknowledgements into the tick in which the count first reaches `min`, since `collect_quorum` emits a slot once. It does **not** fire on the P1b leader-election quorum: `collect_quorum_with_response` selects the *responses* of qualifying keys by `anti_join` against the keys below threshold, provenance does not flow through a negation, and the count's predicate never sits on the records' path; the same holds for any quorum written as "the responses of keys whose count passed". On 2PC (`count >= num_participants`, all-of-n) every output is `unchanged`; echo broadcast has no fold on the path (ablation §"Where it fires in the corpus").

The numbers do not separate it from noise. Paxos bug at cap 256, three repeats: `precise` 23–67, `disjunctive` 21–43, `aggregate` 21–53 hypotheses to the counterexample, draw-dominated. Paxos correct exhausts 256 in every column and every repeat. At cap 400, one pass each, `aggregate` certified at 339 against `precise`'s 393 while `disjunctive` ran out at 400 — within the variance `precise` has shown across this document (203, 230, 256+, 393) — and mean derivation size is unchanged at 19.6–20.4, because the goal's derivation is dominated by leader-election and payload deliveries. 2PC and echo broadcast are identical to their `precise` cells hypothesis for hypothesis; the `hydro_std` quorum unit shrinks its derivations from six deliveries to four at the same 28 hypotheses. Wall time is within noise (ablation §"Paxos", §"Observations" under Aggregate-local witnesses).

It stays off. What would make it worth turning on: a protocol whose quorum predicate sits on the records' path (a count, not an anti-join of responses), whose alternatives appear in the failure-free run rather than only in favourable batchings, and a search long enough for a smaller clause to matter — then a multi-pass measurement with several sampled schedules, since one pass at ±50% draw variance cannot show it.

## 5. Fault point and fault model

The fault point is the sender because the first version put it at the consumer and paid for it. Round 1 used the simulator's scripted `BatchHook` — a standing `OmissionPolicy` on the receiving batch, channels registered by the harness — because interposition there was free. The cost: a `tee`d channel omitted at one consumer was delivered at the others; hook-free paths could not be faulted; crashes needed a separate mechanism in the send half; every faultable batch was a hook parameter in the protocol's signature; and every certificate rested on the author having registered every channel. All followed from one choice: a receiver-side test seam is not a model of the network (history §11). The filter on every `Network` send, with the transport type as the fault model, made every channel a fault point by construction.

The fault model is Molly's after two intermediate positions. The first version admitted crash points at any send; on Raft that "violated" durability on the *correct* protocol when the leader crashed in the heal phase after the body's last election timer had fired. A crash inside the convergence window needs a window of its own, which the body cannot supply without knowing where the crash fell (history §5.4). The search then confined *proposed* crash points to pre-`EFF` sends while the simulator still fired them anywhere — the clamp behind root cause 1 above. Commit `4ddea33f19` closed the gap at the simulator: a crash point named by a pre-`EFF` send fires only if that send happens before `EFF` in the run at hand, and the harness places `EFF` where the failures it means to admit lie. One model, one window, both halves agreeing.

## 6. Monotonicity and what a certificate means

The hitting-set inference — "a fault set that leaves this derivation intact cannot break the view" — is valid only for views monotone in the facts they observe. The generated specification has that form by construction: for a fixed fact and a fixed set of correct members, "every correct member holds this fact" is a positive obligation that another observed `(member, fact)` pair cannot undo, and fixing `FACTS` from the probe prevents a faulty run from minting a new obligation that no earlier lineage explains (hydrofrac.md §"Why the specification's form matters"). Hand-written agreement and durability are not monotone: agreement is the *absence* of a conflict, and durability of a write rests on the *final* leader view. A hypothesis can break either by adding a fact, removing nothing any clause names. Their certificates are graded `RELATIVE`, with no soundness claim for the inference (ldfi.rs `Grade`; history §10).

The concrete lesson was a false certificate. When the observer-level counterfactual landed, `spec_view` took only the replay-derived minimal sufficient set, and on ISR's durability `POST` — the final leader read off `observe_sequenced(views).max()` — deleting views latest-first never changed the value, since the earlier view names the same leader. The sufficient set was the appended write and the *initial* view, with no heartbeat in its lineage; the breaking hypothesis (lose a replication, crash the leader after the acknowledgement, let ZK promote the stale follower) kills neither — it *adds* a later view. Six hypotheses, certificate, bug present. Record provenance, which folds every contribution of an aggregate into its value, kept the views and the heartbeats behind them, and `spec_view` now takes the **union** of the counterfactual and recorded derivations: sound where either is, minimal only where both agree (negation §4). The search finds the corpus's non-monotone bugs because the breaking hypothesis also displaces a recorded fact in `POST`'s derivation; a conflict that appeared without displacing one would not be proposed, and the grade says so.

## 7. The specification is a Hydro flow

`Spec` builds an observer at a dedicated process and compiles it with the program. Observations arrive over an ordinary `fail_stop` channel and are presented in the observer's tick as the set of facts so far; `PRE` and `POST` are bounded `Optional`s in that tick, and a view holds if it is non-empty at the end of the run, as a Datalog goal holds at fixpoint. Because the observer is a Hydro flow its outputs have lineage like any other output, and that is what the search negates (history §6.5). The observer's ticks run eagerly, outside the schedule choice: it only consumes, its interleaving is unobservable to the program, and exploring it multiplied schedules 64× before the rule (history §6.5). For `Spec::all_labels` the observer's input is a slot filled at compile time by an IR pass (`sim::observe`) that lists the labelled collections, turns each labelled node into a `Tee` as a `.clone()` would, maps the second branch to `(collection index, bincode of the element)` and sends it to the observer; the pass runs before network compilation and before the provenance rewrite, so the taps are traced like any operator (hydrofrac.md §"The specification is itself a Hydro flow"). `FACTS` comes from a failure-free probe before the first hypothesis; a probe that yields nothing, or a `PRE` that never holds in the failure-free run, is `Outcome::Vacuous` — a verdict about the

harness, as the 2PC cooperative-termination rows under `all_labels` show when no peer is ever queried in the failure-free run (hydrofrac.md §"Every label of ISR replication, two-phase commit, and ABD").

---

# Part III — Doors

**Closed doors.** Sources: `history-hydrofrac-development.md` (history §n), `history-hydrofrac-negation.md` (negation §n), `ldfi-on-sim-hooks.md` (hooks §"title"), `ldfi-level-ablation.md` (ablation §"title"), and commits.

## Consumer-side simulator hooks as the fault point

- **Tried:** round 1; standing `OmissionPolicy` / `PassthroughPolicy` on the receiving `BatchHook`, channels registered by the harness (hooks §"Prototype status").
- **Closed because:** a test seam, not a network model: `tee`d outputs, hook-free paths, crashes as a separate mechanism, hook parameters in protocol signatures, certificates resting on complete registration (history §11).
- **What it bought us:** the sender-side filter on every `Network` send with the transport type as the fault model; hooks survive as optional lineage precision and as what replay feeds (history §5.5, §11.5).
- **Reopen only if:** never; the sender is where Molly defines omission and crash.

## Crash points named by send count; message identity by occurrence counting

- **Tried:** `crash(node after k sends)` (hooks §"Two soundness problems", 1); `(channel, from, to, payload, k)` with retransmissions as distinct faults (hooks §"Message identity and time").
- **Closed because:** under another hypothesis the send sequence shifts, `send #7` names a different message, and 2PC at load 2 was certified with the bug present.
- **What it bought us:** `crash(node after m)` with `m` identified by `(channel, sender, destination, bytes)`; a node whose `m` is never sent survives; a byte-identical retransmission is the same omission (hydrofrac.md §"Transports supply the fault model"). [NEEDS AUTHOR REASONING: was "an identical retransmission is the same fault" accepted deliberately? A protocol that retries with identical bytes can be tested for "all copies lost" but never for "first lost, retry delivered".]
- **Reopen only if:** a target retransmits byte-identical payloads and the bug needs one copy lost and another delivered.

## Every observed send as a crash candidate

- **Tried:** after message-named crashes, L2 certified ISR at load 2 because the leader's crash *between* two acknowledgements was in no goal derivation; the first fix admitted every observed send (hooks §"Two soundness problems", 2).
- **Closed because:** singleton hypotheses became the number of sends, independent of lineage (2PC load 8: 46 instead of 11).
- **What it bought us:** Molly's answer — the crash right after each send in a `PRE` derivation is a *preserve point*; universe = clause candidates ∪ preserve points.
- **Reopen only if:** never.

## Exploring the observer's schedule

- **Tried:** the observer as an ordinary location whose ticks the explorer interleaved.
- **Closed because:** 64× more schedules on broadcast for an interleaving the program cannot observe (history §6.5).
- **What it bought us:** eager observer ticks outside the schedule choice.
- **Reopen only if:** never.

## The static slice (`Level::Slice`, L1)

- **Tried:** round 1's `sim/slice.rs`, a per-sink backward slice narrowing the execution closure; the default until the collapse (ablation header).
- **Closed because:** L0 = L1 in every cell of every protocol measured (ablation §"Recommendation record"). Removed in commit 4ddea33f19.
- **What it bought us:** the diagnosis that record provenance, not slicing, keeps counts flat under load; `sim/slice.rs` survives to name channels and hook inputs.
- **Reopen only if:** never as a level.

## Counterfactual replay as a level (`Level::Counterfactual`, L3)

- **Tried:** node-local replay as the top rung, falling back to L2 (ablation header).
- **Closed because:** identical derivations to L2 wherever L2 applied, at 2–10× the cost; one win (dataflow ISR bug 3 vs 12) that does not pay for Raft at ~5 s a hypothesis (ablation §"Recommendation record"). Removed in commit 4ddea33f19.
- **What it bought us:** replay as the oracle *inside* `Precise` for opaque records and programs without provenance (commit fafcd70fe3).
- **Reopen only if:** never as a level.

## Downgrading no-provenance programs to `Execution` (`Ldfi::effective_level`)

- **Tried:** commit 4ddea33f19, as part of the collapse.
- **Closed because:** a program with no tags should be *every record opaque*, derived by replay; the downgrade gave `leader_merge` the coarsest lineage and ISR 31 hypotheses instead of 12 (ablation §"Precise without provenance"). Removed in commit fafcd70fe3.
- **What it bought us:** `Precise` as the uniform default; harnesses need not know whether their program carries provenance.
- **Reopen only if:** never.

## Disjunctive provenance on by default

- **Tried:** commit 73a5db9c05, three measurement passes (ablation §"Disjunctive provenance", §"Complete grid" ×2).
- **Closed because:** no reduction on any protocol; +10% / +22% / +40% hypotheses on echo broadcast, because alternative witnesses mint new crash points (ablation §"Diagnostic").
- **What it bought us:** the formula representation (what `Precise` reads its single derivation from); the insight that a lazily grown universe already does Molly's pruning; the DNF-over-single-universe idea.

- **Reopen only if:** a protocol exhibits in-run alternatives whose deliveries are sends the single path already names.

## Aggregate witnesses on by default

- **Tried:** commit 2f156b6335; Paxos × 3 repeats at cap 256, one pass at cap 400.
- **Closed because:** draw-dominated; 339 vs 393 in one pass is within Paxos's ±50% variance; the P1b quorum is unreachable through its `anti_join` (ablation §"Observations" under Aggregate-local witnesses).
- **What it bought us:** which quorum shapes carry recoverable "any `k` of `n`" and which do not.
- **Reopen only if:** see Part II §4.

## The syntactic polarity check

- **Tried:** round 4, `polarity.rs`: a walk over the observer's dataflow grading each view `Monotone` / `NonMonotone` / `Unknown` (negation §5; history §12).
- **Closed because:** it could only approximate (the positive agreements graded `Unknown`; `first()` is an unclassifiable `scan`), and once the specification was generated and sound by form there was nothing to classify. Removed in round 5.
- **What it bought us:** the grades as words — within the fault model for the generated specification, RELATIVE for every hand-written one.
- **Reopen only if:** hand-written specifications become a product rather than an escape hatch.

## Counterfactual-only lineage at the observer

- **Tried:** `spec_view` taking the replay-derived set alone.
- **Closed because:** the false ISR certificate in 6 hypotheses on the final-view `POST` (negation §4).
- **What it bought us:** the union of counterfactual and recorded derivations; the statement that the inference holds only for monotone views.
- **Reopen only if:** the view is monotone by construction, which the generated specification is.

## Clamped crash ordinals; untraced sequenced observations

- **Tried:** found, not tried: the two clause-soundness gaps that certified ISR's bug at L0/L1 and L3 (ablation §"Soundness bug").
- **Closed because:** commit 4c43956b4b made the ordinal exact, moved the clamp to proposal, and taught `payload_matches` the `usize` prefix.
- **What it bought us:** `ldfi_finds_the_unclean_election_bug_at_every_level`; the level grid as a soundness oracle.
- **Reopen only if:** never.

## The staged Raft body ("17 hypotheses")

- **Tried:** rounds 1–4 reported 25, 20, then 17 hypotheses with a body that arranged everything the bug needed except the two lost messages (history §9).
- **Closed because:** the body was part of the claim and its author knew the bug. Withdrawn.
- **What it bought us:** a generic body shaped only by what correct Raft needs; the bug in 90, correct Raft in 235.

- **Reopen only if:** never.

## Tuning the harness body instead of `PRE`

- **Tried:** round 5: four Raft bodies "violated" durability on the correct protocol (no post-`EFF` election timer; no request of the new term; a timer spent on a re-armed suppression; too few heartbeats for the linear back-off), each lengthened; the fourth also changed the protocol (history §9).
- **Closed because:** `PRE` = "some member committed" is true at the first commit, so a run too short to catch up read as a bug: a specification error treated as a scheduling problem (history §1.1).
- **What it bought us:** `PRE` as a convergence witness. The Raft harness is not yet on this pattern (history §12, item 0).
- **Reopen only if:** never; fix `PRE`, not the horizon.

## Silent certification when `PRE` never held

- **Tried:** rounds 1–4: a failure-free run with `PRE` false contributed `{}`, the ILP was infeasible, outcome `Certificate` (history §9).
- **Closed because:** the first generic Raft body sent requests to non-leaders and certified in one run.
- **What it bought us:** `Outcome::Vacuous`.
- **Reopen only if:** never.

## Letting a counterfactual replay panic

- **Tried:** found: Raft's leader indexed past its log under replayed inputs and the panic crossed the dylib boundary and aborted the process (history §9).
- **Closed because:** a replay that cannot run is no evidence, and the abort took the search with it.
- **What it bought us:** `sim::runtime::erase_guarded`; a replay panic is inconclusive and keeps the delivery. Hit 624 times in 365,000 Raft replays.
- **Reopen only if:** never.

## Agreement as the Raft property

- **Tried:** rounds 1–3: "no index holds two committed entries" (hooks §"A Raft bug").
- **Closed because:** agreement is the absence of a conflict; the staged bug was found only because the same omission also displaced a recorded commit (history §6.5, §10).
- **What it bought us:** durability as the Raft property; agreement belongs to the simulator's own tests and Maelstrom.
- **Reopen only if:** never for LDFI.

## A lattice-aware generated specification for `Singleton` outputs

- **Tried:** proposed for ABD's register: observe the final value per correct member, `POST` equality (history §12, item 0).
- **Closed because:** checking the value re-proves the type system's conclusion; the tap goes on the input stream (hydrofrac.md §"Next steps", 1; commit 7d58ad20e0).
- **What it bought us:** a clean class boundary: dissemination, never computation.
- **Reopen only if:** a scalar is labelled over an unlabelled input whose convergence nothing vouches for.

## The undischarged-premise lint

- **Tried:** round 5: every search printed every `EventualConsistency` source and whether the specification observed it (history §1.2).
- **Closed because:** a report cannot fail a test; the direct route was to search every label (auto-labels spec §1).
- **What it bought us:** `Spec::all_labels` / `search_all_labels`; `LabelResults` is the lint.
- **Reopen only if:** never.

## Maelstrom and partition nemeses for the election-restriction bug

- **Tried:** a Raft-backed `lin-kv` workload; twelve runs, 29 minutes, partitions and latency nemeses, ~1,900 elections (history §9.1).
- **Closed because:** the bug needs *selective* loss, and a partition drops everything on a link, so the isolated node falls behind and the index-only check correctly denies it. A premise is discharged by a completeness claim, not by "ran 29 minutes, no anomaly".
- **What it bought us:** the sharp statement of what hydrofrac is for; a real bug in the Maelstrom backend.
- **Reopen only if:** a nemesis drops individual message identities.

## Measurement mistakes

- **Tried:** round 1 charged LDFI 4–8 schedules per hypothesis against one instance per random fault set, and the random baseline visited nodes in registration order, spending its crash budget on the coordinator half the time (hooks §"Ablation").
- **Closed because:** hooks under a policy draw no entropy, so one schedule per hypothesis gave every verdict of eight on protocols without schedule-sensitive bugs; the baseline was lucky, not uninformed.
- **What it bought us:** the corrected grid (lineage beats blind everywhere, 2–7× on certificates); the retraction of "random wins instances-to-bug"; Raft keeps four schedules because its outcome depends on the interleaving (history §5.5). The last round added two rules: wall seconds at load 20–30 are not benchmarks, and $\leq 4$ concurrent exhaustive searches is the ceiling on a 15 GiB host (ablation §"Complete grid: ISR, broadcast, delivery").
- **Reopen only if:** never; but re-measure before quoting a Raft speed-up.

## Why most doors closed

Nearly every entry above fell against one of three walls, and the last round added a fourth.

*The wrong fault point.* A fault injected anywhere but the sender, or named by anything but the message, is not the fault Molly reasons about: its identity shifts between hypotheses, or it misses channels, or it explores a dimension the program cannot see. Consumer-side hooks; crash points by send count and occurrence-counted identity; every observed send as a crash candidate; exploring the observer's schedule; Maelstrom's partition nemeses.

*Unsound inference on a non-monotone view, or on an unsoundly attributed derivation.* Every false certificate came from here. Counterfactual-only lineage at the observer; the polarity check; agreement as the Raft property; the clamped ordinals and untraced observations; letting a replay panic.

*The harness doing the protocol's job.* When the body arranges what the protocol should earn, the verdict is about the author. The staged Raft body; tuning the body instead of `PRE`; silent certification when `PRE` never held.

*Finer provenance does not pay once the search universe is lazy.* The slice, counterfactual as a level, disjunctive provenance and aggregate witnesses all hit this one. Each gives a more exact account of why a run succeeded; none changes which faults the search has reason to try, because hydrofrac proposes a fault only when some derivation depends on the send that names it, and the coarse account already names those sends. Where the finer account named *more* sends (the echo alternatives), the universe grew and the search ran longer for a stronger certificate. The one place finer lineage paid — record provenance over execution lineage under background load — is where the coarse account charged a record to deliveries it did not depend on, and that is already the default. The `effective_level` downgrade is this wall's mirror: coarser lineage where replay was free.

Three entries fit none: the lattice-aware specification and the undischarged-premise lint were about the scope of the claim, and the measurement mistakes about how it was measured.

## Doors never opened

- **Delay and reordering.** Expressible at the filter as hold-instead-of-drop; reordering is the schedule explorer's dimension (history §5.6).
- **Partitions as a primitive.** Omission sets already express them, and partitions miss selective-loss bugs (history §9.1).
- **Byzantine behaviour, duplication.** Outside Molly's vocabulary; duplication has no simulator hook (hydrofrac.md §"Limitations").
- **`cut_after(n)` on a `fail_stop` channel.** Nothing has needed a reliable channel to fail other than by its sender's crash (history §5.6).
- **The filter under `deploy`.** One OS-chosen interleaving per run, no determinism, replay or barriers: Molly's fixed schedule or worse (history §11.3).
- **A receive-side source for hook-free replay.** Would let `BatchHook` drop out of lineage entirely; deferred, since replay reproduces every hooked node and falls back soundly elsewhere (history §11.3).
- **Why-not provenance by Molly's De Morgan rewrite.** The simulator answers the counterfactual question without knowing how the view was computed (negation §2–3).
- **Exhaustive schedules for Raft; blind and random on the generic Raft body; a coverage-guided fuzzer baseline; Raft under load.** Each an hour to a day of compute not yet spent (history §9, §10).
- **Raft `raft-bug` at `Precise` with a 256 cap in the shrunken ablation configuration**, to settle whether the L0 certificate on the bug variant there is a sampled-schedule miss (ablation §"Raft").
- **The disjunction grid on 2PC `n ≥ 3` CTP, `n=4`, Paxos-correct and Raft**, lost to the out-of-memory reboot (ablation §"Complete grid: 2PC, Paxos, leader_merge").

## Open doors

- **Scalar labels checked at their input stream.** A labelled `Singleton` / `Optional` / `KeyedSingleton` is a function of a stream the types already vouch for; the tap goes on the labelling operator's input and the verdict is reported under the scalar's site. ABD's register (`abd.rs:165`) is the first case (hydrofrac.md §"Next steps", 1).
- **Convergence witnesses in `PRE`.** So an unconverged run is vacuous rather than a counterexample; the Raft harness is the first candidate and needs the latest-heartbeat acknowledgement as an output (hydrofrac.md §"Next steps", 2; history §12, 0).

- **Decision-trace replay.** Record every nondeterministic decision and replay a prefix with one changed: the general oracle of which node-local replay is the coarse instance. It does not by itself repair the inference for non-monotone views (negation §3; history §12, 1).
- **Raft's `committed` under the generated specification.** The in-class form of the durability test, with the sound grade; about an hour per search, never run (hydrofrac.md §"Next steps", 4).
- **Membership change.** Nothing on this branch changes configuration while running; the shape is written down — agreement across an epoch boundary, a crash during reconfiguration, membership events as a driven input (history §12, 2).
- **DNF cover constraints over the single-derivation universe.** The one disjunction experiment left (ablation §"Status").
- **Replay cost on long runs.** Caching replays across hypotheses whose scripts share a prefix; a Raft hypothesis late in the certificate search is three to four minutes, almost all replay (history §12, 4).
- **The unmerged ablation knobs.** `LDFI_ABLATION_CTP` / `LDFI_ABLATION_LEVELS` for the 2PC harness (`017e8877af`, bookmark `sandbox-a76a3420-…`) and a parametrised `leader_merge_search` (`0157230103`, bookmark `sandbox-86fabe57-…`) are each one test-only commit on top of `2f156b6335`, written for the disjunction grid the reboot cut short.
- **The old goal API.** `goal`, `goal_all_members`, `goal_witnessed`, `Run::witness` / `require` are compiled and unused by any harness (history §10). [NEEDS AUTHOR REASONING: kept as an escape hatch on purpose, or to be deleted so no new harness is written against it?]

---

## Status and housekeeping

`2f156b6335` is 34 commits past `9d0ec39c92`, a squash whose tree equals `ldfi-hydrofrac` (`7d58ad20e0`, 2026-10-18). Those 34 commits are all of `sim::viz`, the four ablation harnesses, the soundness fix, the Molly fault model, the level collapse and the Precise rename, disjunctive and aggregate provenance, and the ablation document. Anything written against `ldfi-hydrofrac` describes four levels, a `Slice` default, post-`EFF` crash points that fire, and no visualizations. The merge base with `main` (`90d9d26736`) is `99dcd3de68` (2026-08-11); 80 commits on the hydrofrac side, 75 on `main`.

`sandbox-e20764ef-…` (`b56e36ee15`) is a byte-identical tree to `2f156b6335` and can be dropped. `sandbox-86fabe57-…` (`0157230103`) and `sandbox-a76a3420-…` (`017e8877af`) are each one test-only commit on top of `2f156b6335` (above).

The hydrofrac line's first commit (`cd6cae7fc6`, "sandbox init") removed the `sedge`, `sedge_macro`, `cress` and `cress_config_macro` crates that its base (`4de7c45e41`) carried; `main` has none of them either. Whatever branch carries sedge/cress will conflict with the hydrofrac line at the workspace `Cargo.toml` and `Cargo.lock`, and that has to be resolved before either merges toward `main`.

`history-hydrofrac-development.md` carries a one-line header warning that its specifics may not match the code. It should say more loudly that it describes the state before the last round — four levels, `Slice` as default, crash points firing anywhere, `EFF` as "a leftover that should not shape the design" — or be marked archival and left alone. `molly-viz.md` §"Limits" still says alternative derivations are not drawn, which is true of the renderer but no longer of the data under `LDFI_DISJUNCTIVE=1`. The gallery files carry the suffix `record` where a regenerated gallery writes `precise` (gallery README).